

Plan de Pruebas — MusicPlayer

1. Introducción

Este documento describe la estrategia de pruebas aplicada al proyecto MusicPlayer y recoge los casos de prueba ejecutados durante el desarrollo. El objetivo de las pruebas no es solo verificar que el sistema funciona en condiciones ideales, sino también comprobar que se comporta correctamente ante entradas inválidas, permisos insuficientes o condiciones de error externas.

La estrategia elegida combina cuatro niveles de prueba complementarios: pruebas unitarias para la lógica de los servicios Angular, pruebas de integración para los controladores REST del backend, pruebas manuales funcionales para los flujos de usuario completos y pruebas de regresión visual para la consistencia del diseño. Esta combinación permite detectar errores a distintos niveles de granularidad y con distintos grados de automatización.

Un criterio de diseño importante en este plan fue que las pruebas de integración del backend deben ejecutarse contra una base de datos real (H2 en memoria configurada para tests), y no contra mocks de repositorios. Esta decisión surgió de la necesidad de verificar que las consultas JPA son correctas, que las restricciones de clave foránea se respetan y que el comportamiento de Liquibase en el contexto de test es el esperado. Usar mocks de repositorios habría ocultado exactamente el tipo de errores que más interesaba detectar.

2. Estrategia de Pruebas

2.1 Tipos de prueba

Tipo	Herramienta	Alcance
Pruebas unitarias (frontend)	Vitest 4.x	Servicios Angular, lógica de componentes
Pruebas de integración (backend)	JUnit 5 + Spring Boot Test	Controladores REST, repositorios JPA
Pruebas manuales funcionales	Navegador + Postman	Flujos de usuario completos
Pruebas de regresión visual	Manual en Chrome/Firefox	Consistencia del diseño tras cambios

2.2 Niveles de prueba

La pirámide de pruebas del proyecto está organizada de la siguiente forma:

- **Base (pruebas automáticas del backend):** las pruebas de integración de JUnit 5 cubren los controladores REST, verificando el código de respuesta HTTP, el cuerpo de la respuesta y el control de acceso por rol. Son las pruebas que más confían da aportan porque arrancan el contexto completo de Spring Boot con una base de datos H2 en memoria.
- **Nivel medio (pruebas unitarias del frontend):** los servicios Angular se prueban con Vitest de forma aislada, sin necesidad de arrancar el servidor backend. Cubren la lógica de transformación de datos, el manejo de errores HTTP y el comportamiento de las señales reactivas.
- **Cima (pruebas manuales funcionales):** los flujos de usuario completos se verifican manualmente en el navegador, siguiendo los casos de prueba documentados en este plan. Son las pruebas más lentas pero las que más se acercan a la experiencia real del usuario.

2.3 Entorno de pruebas

Las pruebas del frontend se ejecutan contra el servidor de desarrollo Angular en `http://localhost:4200` , con el backend Spring Boot corriendo en `http://localhost:8080` y la base de datos MySQL levantada en Docker en `localhost:3306` . Para las pruebas de integración del backend se usa H2 en memoria, de forma que cada ejecución parte de un estado limpio y reproducible.

Las credenciales utilizadas durante las pruebas manuales son las siguientes:

- **Administrador:** `admin / admin`
- **Oyente:** `user / user`

Para los casos de prueba que requieren el rol EDITOR o ARTIST, se creó previamente un usuario con cada rol mediante la gestión de usuarios del administrador.

2.4 Criterios de aceptación

Un caso de prueba se considera superado cuando el comportamiento observado del sistema coincide exactamente con el resultado esperado documentado en la columna "Resultado esperado". Un caso de prueba se considera fallido cuando el comportamiento difiere del esperado, independientemente de si el sistema produce un error visible para el usuario o simplemente hace algo distinto de lo que debería.

Los casos de prueba fallidos se registran con la descripción del comportamiento observado, el log de error si existe y el módulo afectado. No se cierra la ejecución de pruebas de un módulo hasta que todos sus casos de prueba han pasado.

3. Casos de Prueba

3.1 Módulo de Autenticación

La autenticación es la puerta de entrada al sistema y el componente de mayor criticidad desde el punto de vista de la seguridad. Un fallo en este módulo podría comprometer todo el control de acceso del sistema, por lo que se diseñaron casos de prueba que cubren tanto el camino feliz (credenciales válidas) como los casos negativos más importantes (credenciales incorrectas, tokens expirados, acceso sin token, acceso con rol insuficiente).

ID	Descripción	Pasos	Resultado esperado	Resultado
AUTH-01	Login con credenciales válidas	1. Ir a <code>/login</code> 2. Introducir <code>admin/admin</code> 3. Pulsar "Iniciar sesión"	Redirige al dashboard de administrador	✅ Pasa
AUTH-02	Login con credenciales inválidas	1. Ir a <code>/login</code> 2. Introducir usuario/contraseña incorrectos 3. Pulsar "Iniciar sesión"	Muestra mensaje de error, no redirige	✅ Pasa
AUTH-03	Logout	1. Estar autenticado 2. Pulsar "Cerrar sesión" en el navbar	Redirige a <code>/login</code> , token eliminado de <code>localStorage</code>	✅ Pasa
AUTH-04	Acceso a ruta protegida sin token	1. Sin autenticar, escribir directamente <code>/home</code> en la barra de direcciones	<code>AuthGuard</code> redirige a <code>/login</code>	✅ Pasa
AUTH-05	Acceso a ruta de admin con rol usuario	1. Autenticar como <code>user/user</code> 2. Navegar a <code>/admin</code>	Muestra pantalla de acceso denegado	✅ Pasa

AUTH-06	Persistencia de sesión tras refresco	1. Autenticar correctamente 2. Refrescar la página (F5)	El usuario sigue autenticado; el token sigue en localStorage	✓ Pasa
AUTH-07	Login con campo contraseña vacío	1. Ir a <code>/login</code> 2. Introducir solo el usuario, dejar contraseña vacía 3. Pulsar "Iniciar sesión"	El formulario muestra validación de campo requerido antes de enviar la petición	✓ Pasa
AUTH-08	Login con campo usuario vacío	1. Ir a <code>/login</code> 2. Dejar usuario vacío, introducir contraseña 3. Pulsar "Iniciar sesión"	El formulario muestra validación de campo requerido	✓ Pasa

El caso AUTH-04 verifica el comportamiento del `AuthGuard` de Angular, que intercepta la navegación antes de que el componente de destino se cargue. El caso AUTH-05 verifica el guard de roles, que comprueba no solo que el token existe sino también que contiene el rol requerido por la ruta. El caso AUTH-06 verifica que el token se persiste correctamente en `localStorage` y que el servicio `AccountService` lo recupera al inicializar la aplicación.

3.2 Módulo de Dashboards por Rol

Uno de los requisitos más relevantes de MusicPlayer es que la experiencia de usuario varía en función del rol. No se trata únicamente de restringir acciones: el punto de entrada (el dashboard), la navegación lateral y los accesos directos disponibles son distintos para cada perfil. Esta diferenciación tiene que ser correcta en todos los roles, sin que el cambio de rol de un usuario provoque efectos inesperados en el componente `HomeComponent`.

ID	Descripción	Pasos	Resultado esperado	Resultado
DASH-01	Dashboard administrador	Autenticar como <code>admin/admin</code> , navegar a <code>/home</code>	Se muestra el dashboard de administrador con métricas y accesos de gestión	✓ Pasa
DASH-02	Dashboard editor	Autenticar como usuario con rol EDITOR, navegar a <code>/home</code>	Se muestra el catálogo musical y herramientas de edición	✓ Pasa
DASH-03	Dashboard oyente	Autenticar como <code>user/user</code> , navegar a <code>/home</code>	Se muestran las listas de reproducción y el historial del usuario	✓ Pasa
DASH-04	Sidebar adaptada por rol	Autenticar con distintos roles sucesivamente	Los enlaces del sidebar varían según el rol: admin ve gestión, editor ve catálogo, oyente ve playlists	✓ Pasa
DASH-05	El oyente no ve opciones de gestión	Autenticar como <code>user/user</code> , inspeccionar el sidebar	No aparecen enlaces a gestión de canciones, álbumes, artistas ni usuarios	✓ Pasa

3.3 Módulo de Gestión de Canciones

La gestión de canciones es el núcleo del catálogo musical. Las operaciones CRUD deben funcionar correctamente para los roles autorizados, y los usuarios sin permisos de escritura no deben ver ni poder acceder a las opciones de

edición. Este módulo es representativo del resto de módulos de entidades (álbumes, artistas, géneros), que siguen exactamente el mismo patrón.

ID	Descripción	Pasos	Resultado esperado	Resultado
SONG-01	Listar canciones	Navegar a <code>/song</code> con cualquier rol autenticado	Se muestra la tabla de canciones con paginación y cabecera <code>X-Total-Count</code>	✓ Pasa
SONG-02	Crear canción (editor)	Como EDITOR, ir a <code>/song/new</code> , rellenar todos los campos obligatorios y guardar	Canción creada con código 201, visible en la lista al regresar	✓ Pasa
SONG-03	Editar canción	Como EDITOR, pulsar el icono de edición de una canción, modificar el título y guardar	Los cambios se reflejan en la lista y en el detalle de la canción	✓ Pasa
SONG-04	Eliminar canción	Como EDITOR, pulsar el icono de eliminación y confirmar en el diálogo	La canción desaparece de la lista y el servidor devuelve 204	✓ Pasa
SONG-05	Ver detalle de canción	Pulsar sobre el nombre de una canción en la lista	Se muestra la página de detalle con todos los campos: título, álbum, artistas, género, duración, portada	✓ Pasa
SONG-06	Usuario sin permisos no ve botones de edición	Autenticar como <code>user/user</code> , navegar a <code>/song</code>	La tabla de canciones se muestra, pero no aparecen los botones de crear, editar ni eliminar	✓ Pasa
SONG-07	Crear canción sin campos obligatorios	Como EDITOR, intentar guardar una canción sin título	El formulario muestra un mensaje de validación; no se realiza la petición POST	✓ Pasa
SONG-08	Paginación de la lista de canciones	Navegar a <code>/song</code> con más de 20 canciones en el catálogo	Se muestra la primera página con el número correcto de registros; el componente de paginación refleja el total	✓ Pasa
SONG-09	Acceso directo a endpoint de creación sin rol	Enviar POST a <code>/api/songs</code> con token de <code>user/user</code> desde Postman	El backend devuelve 403 Forbidden	✓ Pasa
SONG-10	Acceso a endpoint sin token	Enviar GET a <code>/api/songs</code> sin cabecera Authorization desde Postman	El backend devuelve 401 Unauthorized	✓ Pasa

El caso SONG-06 es especialmente importante: verifica que el control de acceso en el frontend no es solo una restricción de navegación, sino que también oculta los elementos de la interfaz que no son relevantes para el rol del

usuario. Los casos SONG-09 y SONG-10 comprueban el control de acceso a nivel de backend, independientemente de lo que haga el frontend.

3.4 Módulo del Reproductor

El reproductor visual es el componente más visible de la interfaz. Está fijo en la parte inferior de la pantalla y siempre accesible, independientemente de la página en la que se encuentre el usuario. Sus controles deben responder correctamente a las acciones del usuario y el estado visual debe ser consistente en todo momento.

ID	Descripción	Pasos	Resultado esperado	Resultado
PLAY-01	Barra del reproductor visible en todas las páginas	Navegar entre distintas páginas tras login	El player bar permanece fijo en la parte inferior en todas las rutas	✓ Pasa
PLAY-02	Toggle reproducir/pausar	Pulsar el botón central del reproductor	El icono alterna entre play (triángulo) y pause (dos barras); el estado <code>isPlaying</code> cambia	✓ Pasa
PLAY-03	Toggle aleatorio	Pulsar el botón de aleatorio	El botón cambia de color neutro a color de acento, indicando que el modo está activo	✓ Pasa
PLAY-04	Toggle repetir	Pulsar el botón de repetición	El botón cambia a color de acento en estado activo	✓ Pasa
PLAY-05	Control de volumen	Mover el slider de volumen	La barra de relleno del slider refleja el nuevo valor; el estado de volumen se actualiza	✓ Pasa
PLAY-06	Silenciar/des-silenciar	Pulsar el icono de volumen	El icono alterna entre volumen normal y silenciado; el slider baja a 0 y al des-silenciar recupera el valor anterior	✓ Pasa
PLAY-07	Barra de progreso	Mover el slider de progreso	La barra de relleno refleja la posición seleccionada de forma visual	✓ Pasa
PLAY-08	Estado inicial del reproductor	Acceder a la aplicación sin haber seleccionado ninguna canción	El reproductor muestra un estado neutral: sin nombre de canción, botones desactivados visualmente	✓ Pasa
PLAY-09	Reproducción real de audio	Seleccionar una canción que tiene <code>fileUrl</code> configurado, pulsar Play	El navegador reproduce el audio real; la barra de progreso avanza en tiempo real	✓ Pasa

PLAY-10	Salto de progreso durante reproducción	Con una canción reproduciéndose, arrastrar el slider de progreso a mitad	El audio salta al punto seleccionado y continúa reproduciéndose desde ahí	✓ Pasa
PLAY-11	Botón Anterior con <3s de reproducción	Pulsar Anterior cuando la canción lleva menos de 3 segundos	Va a la canción anterior de la cola	✓ Pasa
PLAY-12	Botón Anterior con >3s de reproducción	Pulsar Anterior cuando la canción lleva más de 3 segundos	Vuelve al inicio de la canción actual (no va a la anterior)	✓ Pasa
PLAY-13	Cola de reproducción — siguiente	Con varias canciones en cola, pulsar Siguiente varias veces	El reproductor avanza por las canciones en orden y muestra el nombre correcto	✓ Pasa
PLAY-14	Modo aleatorio cambia el orden	Con shuffle activo, pulsar Siguiente repetidamente	Las canciones se reproducen en orden aleatorio (no siempre la misma secuencia)	✓ Pasa
PLAY-15	Modo repetición	Con repeat activo, esperar a que acabe la canción	La canción vuelve a reproducirse desde el principio automáticamente	✓ Pasa

Todos los estados del reproductor están gestionados con señales Angular, lo que garantiza que cualquier cambio en el estado (por ejemplo, silenciar) se propaga automáticamente a todos los elementos de la vista que dependen de ese estado, sin necesidad de disparar manualmente la detección de cambios.

3.5 Módulo de Letras de Canciones

Las letras de canciones se obtienen de la API externa `lyrics.ovh`. Este módulo tiene la particularidad de depender de un servicio de terceros, lo que implica que los casos de prueba deben cubrir no solo el camino feliz sino también los escenarios de error: letras no encontradas y fallo de red.

ID	Descripción	Pasos	Resultado esperado	Resultado
LYR-01	Mostrar panel de letras	Pulsar el botón de letras en el player bar	El panel se despliega encima del reproductor con una animación suave	✓ Pasa
LYR-02	Ocultar panel de letras	Con el panel abierto, pulsar el botón de letras de nuevo	El panel se cierra	✓ Pasa
LYR-03	Indicador de carga	Pulsar el botón de letras (con canción cargada)	Aparece un spinner mientras se realiza la petición a <code>lyrics.ovh</code>	✓ Pasa
LYR-04	Letras encontradas	Seleccionar una canción con artista y título conocidos (p. ej. "Ed Sheeran - Shape of You")	Se muestra el texto completo de la letra en el panel	✓ Pasa

LYR-05	Letras no encontradas	Usar una canción con datos ficticios o sin resultados en la API	Se muestra el mensaje: "No se encontraron letras para esta canción."	✓ Pasa
LYR-06	Error de red	Simular desconexión de red (DevTools > Network > Offline) antes de pulsar el botón de letras	Se muestra un mensaje de error amigable sin romper el resto de la interfaz	✓ Pasa

El caso LYR-06 requiere el uso de las herramientas de desarrollo del navegador para simular la desconexión. Es importante verificarlo porque un error de red no gestionado podría provocar que el componente quedase en estado de carga indefinido o que lanzase un error no capturado en la consola. `LyricsService` usa el operador `catchError` de RxJS para transformar cualquier error de la petición HTTP en un estado de error controlado.

3.6 Módulo de Listas de Reproducción

Las playlists son una funcionalidad central para el rol oyente. El sistema debe permitir crear playlists, añadir canciones a ellas y controlar su visibilidad (pública/privada). También debe garantizar que las playlists privadas no son visibles para usuarios distintos al propietario.

ID	Descripción	Pasos	Resultado esperado	Resultado
PLIST-01	Crear lista de reproducción	Como USER, navegar a <code>/playlist/new</code> , rellenar el nombre y guardar	La playlist se crea y aparece en la lista del usuario	✓ Pasa
PLIST-02	Ver canciones de una playlist	Abrir el detalle de una playlist con canciones asociadas	Se listan las canciones con su posición, título y álbum	✓ Pasa
PLIST-03	Añadir canción a playlist	Desde la gestión de <code>PlaylistSong</code> , seleccionar playlist y canción y guardar	La canción aparece en la playlist en la posición indicada	✓ Pasa
PLIST-04	Playlist pública vs privada	Crear una playlist con <code>isPublic: false</code> y acceder a ella con un usuario distinto	El usuario distinto no puede ver ni acceder a la playlist privada	✓ Pasa
PLIST-05	Eliminar playlist	Como USER, pulsar el botón de eliminación en una playlist propia y confirmar	La playlist desaparece de la lista; el servidor devuelve 204	✓ Pasa
PLIST-06	No se puede editar playlist ajena	Intentar enviar PUT a <code>/api/playlists/{id}</code> con un id de playlist de otro usuario	El backend devuelve 403 Forbidden	✓ Pasa

3.7 Módulo de Artistas y Álbumes

Artistas y álbumes son entidades de referencia del catálogo. Los artistas pueden tener el campo `verified` que indica que han sido verificados por la plataforma. Los álbumes tienen un tipo (`ALBUM` , `SINGLE` , `EP` , `PODCAST_SERIES`) que permite clasificar el contenido.

ID	Descripción	Pasos	Resultado esperado	Resultado
ART-01	Listar artistas	Navegar a <code>/artist</code> con cualquier rol autenticado	Se muestra la tabla de artistas paginada con nombre, país y estado de verificación	✓ Pasa
ART-02	Campo artista verificado	Como EDITOR, crear un artista con <code>verified: true</code>	El detalle del artista muestra el campo como verificado	✓ Pasa
ART-03	Editar artista	Como EDITOR, modificar el nombre de un artista existente y guardar	El cambio se refleja en la lista y en el detalle del artista	✓ Pasa
ART-04	Eliminar artista sin canciones asociadas	Como ADMIN, eliminar un artista que no tiene canciones	El artista se elimina correctamente; el servidor devuelve 204	✓ Pasa
ALB-01	Crear álbum	Como EDITOR, crear un álbum con tipo <code>ALBUM</code> y asociarlo a un artista	El álbum se crea y aparece en la lista con el artista y tipo correctos	✓ Pasa
ALB-02	Tipos de álbum	Como EDITOR, abrir el formulario de creación de álbum	El selector de tipo muestra exactamente cuatro opciones: <code>ALBUM, SINGLE, EP, PODCAST_SERIES</code>	✓ Pasa
ALB-03	Detalle de álbum con canciones	Navegar al detalle de un álbum que tiene canciones	Se muestran las canciones del álbum con su título y duración	✓ Pasa
ALB-04	Álbum no puede crearse sin artista	Intentar guardar un álbum sin seleccionar artista	El formulario muestra un error de validación; no se envía la petición	✓ Pasa

3.8 Módulo de Géneros

Los géneros son la clasificación más básica del catálogo. Son necesarios para categorizar canciones y álbumes. La gestión de géneros sigue el mismo patrón CRUD que el resto de entidades del catálogo.

ID	Descripción	Pasos	Resultado esperado	Resultado
GENRE-01	Listar géneros	Navegar a <code>/genre</code> con cualquier rol autenticado	Se muestra la lista de géneros disponibles	✓ Pasa
GENRE-02	Crear género	Como EDITOR, crear un nuevo género con nombre único	El género se crea y aparece en la lista	✓ Pasa
GENRE-03	Editar género	Como EDITOR, modificar el nombre de un género y guardar	El cambio se refleja en la lista	✓ Pasa
GENRE-04	Eliminar género sin canciones asociadas	Como ADMIN, eliminar un género sin canciones ni	El género se elimina; el servidor devuelve	✓ Pasa

		álbumes vinculados	204	
--	--	--------------------	-----	--

3.9 Módulo de Reproducciones e Historial

El historial de reproducciones registra cuándo y qué canciones ha escuchado cada usuario. Es la base para funcionalidades futuras como el sistema de recomendación. Las pruebas verifican que el historial se registra correctamente y que cada usuario solo puede ver el suyo.

ID	Descripción	Pasos	Resultado esperado	Resultado
HIST-01	Registrar reproducción	Enviar POST a <code>/api/plays</code> con id de canción válido	El backend devuelve 201 Created con la reproducción registrada	✓ Pasa
HIST-02	Ver historial propio	Como USER, navegar al historial de reproducciones	Se muestra la lista de canciones escuchadas con fecha y hora	✓ Pasa
HIST-03	No se puede ver historial ajeno	Enviar GET a <code>/api/plays</code> filtrado por otro <code>userId</code> desde Postman	El backend devuelve solo las reproducciones del usuario autenticado, ignorando el filtro	✓ Pasa
HIST-04	Eliminar entrada del historial	Como USER, eliminar una entrada del historial	La entrada desaparece de la lista; el servidor devuelve 204	✓ Pasa

3.10 Módulo de Favoritos

El módulo de favoritos permite a los usuarios marcar canciones que quieren guardar para encontrarlas rápidamente. Es una funcionalidad simple pero que implica una relación entre usuario y canción que debe gestionarse correctamente.

ID	Descripción	Pasos	Resultado esperado	Resultado
LIKE-01	Marcar canción como favorita	Como USER, pulsar el botón de favorito en una canción (toggle)	El icono de favorito cambia de estado; el servidor llama a <code>POST /api/likes/toggle/{songId}</code>	✓ Pasa
LIKE-02	Ver canciones favoritas	Como USER, navegar a la sección de favoritos	Se muestra la lista de canciones marcadas como favoritas	✓ Pasa
LIKE-03	Desmarcar favorito	Como USER, pulsar el botón de favorito en una canción ya marcada	El icono vuelve al estado neutro; la canción desaparece de la lista de favoritos	✓ Pasa
LIKE-04	No se pueden ver favoritos ajenos	Enviar GET a <code>/api/likes</code> con token de otro usuario	El backend devuelve solo los favoritos del usuario autenticado	✓ Pasa

3.11 Módulo de Subida de Archivos

La subida de archivos es fundamental para añadir contenido al catálogo. El backend valida el tipo de archivo (imagen o audio), genera un nombre único y lo almacena localmente. El endpoint de streaming permite reproducirlo directamente desde el reproductor.

ID	Descripción	Pasos	Resultado esperado	Resultado
UPLOAD-01	Subir imagen de portada	Como EDITOR, en el formulario de canción, subir un archivo JPG de menos de 10 MB	El servidor devuelve 200 con la URL del archivo; la portada se muestra en el formulario	✅ Pasa
UPLOAD-02	Subir audio MP3	Como EDITOR, en el formulario de canción, subir un archivo MP3 de menos de 10 MB	El servidor devuelve 200 con la URL y el nombre del archivo; el campo <code>fileUrl</code> se rellena	✅ Pasa
UPLOAD-03	Streaming de audio	Con una canción que tiene <code>fileUrl</code> configurado, pulsar Play en el reproductor	El navegador obtiene el audio de <code>/api/upload/stream/{filename}</code> y lo reproduce	✅ Pasa
UPLOAD-04	Subir archivo que supera el límite	Intentar subir un archivo de más de 10 MB	El servidor devuelve 400 Bad Request; el formulario muestra un mensaje de error	✅ Pasa
UPLOAD-05	Subir tipo incorrecto (imagen donde audio)	En el campo de audio, subir un archivo PDF	El servidor devuelve 400 Bad Request con mensaje "Solo se permiten archivos de audio"	✅ Pasa

3.12 Módulo de Búsqueda

El buscador permite encontrar canciones por título o artista. La búsqueda es reactiva: los resultados se actualizan en tiempo real y la URL refleja el término buscado, lo que permite compartir búsquedas mediante enlace.

ID	Descripción	Pasos	Resultado esperado	Resultado
SEARCH-01	Búsqueda con resultados	Introducir un término que coincide con el título de una canción conocida	Se muestran los resultados relevantes con nombre del artista y duración	✅ Pasa
SEARCH-02	Búsqueda sin resultados	Introducir un término que no coincide con ninguna canción	Se muestra el estado vacío con un mensaje apropiado	✅ Pasa

SEARCH-03	Reproducir desde búsqueda	Pulsar Play en un resultado de búsqueda	La canción se reproduce y el resto de resultados forma la cola de reproducción	✅ Pasa
SEARCH-04	URL de búsqueda enlazable	Navegar directamente a <code>/search?q=extremoduro</code>	La búsqueda se ejecuta automáticamente y muestra los resultados al cargar la página	✅ Pasa

4. Pruebas de Regresión Visual

La regresión visual es uno de los riesgos más habituales en proyectos con un diseño elaborado: un cambio en el SCSS de un componente puede afectar inesperadamente a otros componentes que comparten estilos. Para mitigarlo, se realizaron comprobaciones visuales manuales tras cada cambio significativo en los archivos de estilos.

Las comprobaciones se realizaron en dos navegadores (Chrome y Firefox) y en tres tamaños de pantalla representativos: 375 píxeles de ancho (móvil), 768 píxeles (tableta) y 1280 píxeles o más (escritorio). En cada comprobación se verificaron los siguientes aspectos:

Consistencia del tema oscuro. Todas las páginas deben usar los mismos tonos de fondo oscuro definidos en las variables SCSS (`$bg-primary` , `$bg-secondary`). Un error típico es que un componente use un color de fondo con valor fijo en lugar de la variable, lo que hace que no respete el tema si las variables cambian.

Solapamiento entre sidebar, navbar y player bar. Estos tres elementos de maquetación están posicionados de forma fija o sticky en distintos bordes de la pantalla. El contenido principal debe tener los márgenes correctos para no quedar oculto detrás de ninguno de ellos. Este punto fue especialmente problemático en pantallas pequeñas, donde los márgenes reducidos pueden provocar que el contenido quede tapado por la player bar.

Legibilidad del texto. El contraste entre el texto y el fondo debe ser suficiente en todos los elementos. En particular, el texto secundario (color `$text-secondary: #b3b3b3`) sobre el fondo principal (`$bg-primary: #0f172a`) debe mantener un ratio de contraste mínimo para cumplir con las recomendaciones de accesibilidad WCAG 2.1 AA.

Responsividad de tablas y formularios. Las tablas de entidades y los formularios de creación/edición deben ser usables en pantallas estrechas, con scroll horizontal donde sea necesario, sin que el contenido se desborde del viewport.

Alineación y espaciado. Los elementos de la interfaz deben mantener un espaciado consistente entre páginas. Un padding incorrecto en un componente puede hacer que el conjunto de la página no tenga el aspecto esperado aunque cada elemento individual sea correcto.

5. Pruebas de Integración del Backend

Las pruebas de integración del backend se ubican en `src/test/java/com/musicplayer/` . Estas pruebas arrancan el contexto completo de Spring Boot (en modo test, con H2 en memoria) y ejecutan peticiones HTTP reales contra los controladores REST.

El enfoque de estas pruebas cubre cuatro áreas principales:

Autenticación y generación de JWT. Se verifica que `AuthenticateController` devuelve un token válido para credenciales correctas y un error 401 para credenciales incorrectas. También se verifica que el token generado contiene el rol correcto y que su fecha de expiración es la esperada.

Operaciones CRUD. Para cada controlador REST principal se prueba la creación (POST), la lectura paginada (GET), la lectura por ID (GET/{id}), la actualización (PUT) y la eliminación (DELETE) de recursos. Cada prueba verifica el código HTTP de respuesta y el cuerpo de la respuesta mediante la deserialización del JSON resultante.

Validación de restricciones. Se verifica que el backend devuelve un error 400 con un mensaje descriptivo cuando se intenta crear una entidad con campos obligatorios vacíos o con valores fuera de rango. Por ejemplo, crear una canción sin título debe devolver 400 con un mensaje que indique el campo que falla.

Control de acceso. Se verifica que los endpoints protegidos devuelven 401 sin token, 403 con token de rol insuficiente y 200/201/204 con el rol correcto. Estas pruebas son especialmente importantes porque el frontend también implementa control de acceso, pero la capa de backend es la que realmente garantiza la seguridad del sistema.

Para ejecutar las pruebas de integración:

```
./mvnw verify
```

JaCoCo genera un informe de cobertura en `target/site/jacoco/index.html` tras la ejecución.

Para ejecutar solo las pruebas unitarias del frontend:

```
npm test
```

Y para el linting del código frontend:

```
npm run lint
```

6. Pruebas de Usabilidad

Además de las pruebas funcionales y técnicas, se realizaron pruebas de usabilidad informales con tres usuarios voluntarios que no habían interactuado con el sistema anteriormente. El objetivo de estas pruebas fue identificar puntos de fricción en la interfaz que no habían sido detectados durante el desarrollo, donde tanto el desarrollador del frontend como el del backend conocen la aplicación en profundidad y pueden anticipar los flujos de uso.

6.1 Metodología

Las pruebas de usabilidad se realizaron de forma presencial. A cada usuario se le pidió que realizase cinco tareas sin recibir ayuda del evaluador:

1. Registrarse en la plataforma con una cuenta nueva
2. Navegar a la lista de canciones y abrir el detalle de una canción
3. Crear una nueva lista de reproducción
4. Abrir el panel de letras de una canción desde el reproductor
5. Cerrar sesión

Durante cada tarea, el evaluador observó el comportamiento del usuario sin intervenir, anotando los puntos donde el usuario dudaba, se equivocaba o necesitaba más tiempo del esperado. Al finalizar las cinco tareas, se pidió a cada usuario que respondiese el cuestionario SUS (System Usability Scale) para obtener una valoración cuantitativa de la usabilidad percibida.

6.2 Cuestionario SUS

El cuestionario SUS consta de diez afirmaciones que el usuario valora en una escala de 1 (totalmente en desacuerdo) a 5 (totalmente de acuerdo):

#	Afirmación
1	Me gustaría usar este sistema con frecuencia
2	El sistema me pareció innecesariamente complejo
3	El sistema me pareció fácil de usar
4	Necesitaría apoyo de un técnico para usar este sistema
5	Las diferentes funciones de este sistema están bien integradas
6	El sistema me pareció muy inconsistente
7	La mayoría de las personas aprenderían a usar este sistema muy rápidamente
8	El sistema me resultó muy incómodo de usar
9	Me sentí con confianza al usar el sistema
10	Necesitaba aprender muchas cosas antes de poder empezar a usar el sistema

La puntuación SUS se calcula sumando las contribuciones de cada ítem. Para los ítems impares, la contribución es (valor_usuario - 1). Para los ítems pares, la contribución es (5 - valor_usuario). La suma total se multiplica por 2,5 para obtener una puntuación entre 0 y 100.

6.3 Resultados de las pruebas de usabilidad

Usuario	SUS Score	Observaciones principales
Usuario 1	77,5	Dudó en la tarea de registro (no encontraba el enlace al formulario en el primer intento). Una vez dentro, la navegación le resultó intuitiva.
Usuario 2	82,5	Completó todas las tareas sin problemas. Comentó que le gustaba el aspecto visual. Tardó algo más en encontrar el botón de letras en el reproductor.
Usuario 3	72,5	Tuvo dificultades con la tarea de crear una playlist (buscaba la opción en el sidebar en lugar de en el módulo de playlists). Sugirió añadir un acceso directo en el sidebar.

La puntuación SUS media obtenida fue de **77,5 puntos**, que se sitúa en la categoría "Bueno" según la escala de clasificación de Bangor et al. (2008). Los puntos de mejora identificados son la visibilidad del enlace de registro en la página de login y la accesibilidad del botón de letras en el reproductor, que es pequeño y puede pasar desapercibido.

7. Resultados y Valoración Global

Los casos de prueba documentados en este plan resultaron exitosos en la versión actual del sistema. Los problemas encontrados durante las pruebas se corrigieron antes de cerrar la ejecución de cada módulo, por lo que no hay casos pendientes ni bloqueantes.

El módulo con más incidencias durante las pruebas fue el de la sidebar y el reproductor en diseño responsive. La coexistencia de tres elementos de maquetación fijos (navbar, sidebar, player bar) en pantallas de 375 píxeles de ancho requirió varias iteraciones de ajuste de márgenes y z-index hasta conseguir un resultado correcto en todos los tamaños.

El módulo de letras de canciones presentó un comportamiento intermitente en el caso LYR-04, ya que la disponibilidad de `lyrics.ovh` no es garantizada. En algunos momentos de las pruebas el servicio tardó más de cinco segundos en responder, lo que hizo que el spinner de carga permaneciese visible durante un tiempo incómodo para el usuario. Este comportamiento está documentado como área de mejora: en una versión futura, se podría añadir un timeout explícito de dos o tres segundos tras el cual se muestre un mensaje de error sin esperar indefinidamente.

En cuanto a las pruebas de usabilidad, la puntuación SUS de 77,5 confirma que la interfaz es percibida como fácil de usar por usuarios nuevos, aunque hay puntos concretos de mejora identificados que podrían elevarse hasta la categoría "Excelente" ($SUS \geq 85$) en versiones futuras.

En conjunto, la combinación de pruebas unitarias, de integración, manuales funcionales y de usabilidad proporcionó un nivel de confianza razonable en la corrección del sistema para los flujos de usuario documentados.